Physics-Informed Neural Networks for Cantilever Beam Parameter Identification

Version History and Concept Reference

v6 → v7 → v8 → v8.1

This document is a self-contained reference for the PINN-based inverse identification project. It records (i) the exact change set between each version, (ii) detailed explanations of every concept the method relies on (spectral bias, Fourier features, loss normalisation, curriculum learning, the trivial-solution failure mode, bending vs. torsional modes, SA-PINNs), and (iii) the key questions raised during development and how they were resolved. A reader with a computer-science background and no prior structural dynamics should be able to follow the whole pipeline end to end.

Generated as a project working document.

Contents

1 Purpose and How to Read This Document

The project identifies five physical quantities of a slender cantilever beam from a single experimental velocity frequency-response function (FRF), measured at the beam tip. Four of the unknowns are scalars; the fifth, the Young's modulus $E(x)$, is a *function* of position along the beam. That function-valued unknown is the entire reason a Physics-Informed Neural Network (PINN) is used instead of classical curve-fitting.

The work proceeds in versioned Jupyter notebooks. Each version is a small, deliberate change over the previous one — the working discipline is to copy prior cells verbatim and modify only the cells whose logic actually changes. This document explains that chain of changes and the reasoning behind each one.

Sections 2–3 establish the fixed parts of the method: the physics and the neural-network machinery that did *not* change. Section 4 is the version-by-version change log. Section 5 explains every concept in depth. Section 6 is a log of the questions raised during development and the clarifications that resolved them. Section 7 is a practical runbook. Section 8 is a symbol glossary.

2 The Physical Problem

2.1 The beam and what is measured

A thin metal strip is rigidly clamped at one end ($x = 0$) and free at the other ($x = L$). A known small mass is attached at the tip. The beam is excited harmonically near the tip, and the resulting *velocity* is measured at the tip across a range of frequencies. The raw measurement is a complex-valued velocity FRF: for each excitation frequency ω, a complex number whose magnitude and phase describe how strongly and with what delay the tip moves.

Quantity	Symbol	Value
Beam length	L	126.06 mm
Width	b	10.13 mm
Thickness	h	0.95 mm
Density	ρ	8216 kg/m^3
Tip mass	m_{tip}	10 g
Cross-section area	$A = bh$	derived
Second moment of area (weak axis)	$I = bh^3/12$	derived

The cross-section is very wide relative to its thickness ($b/h \approx 10.7$), i.e. a thin wide strip. This matters later: such cross-sections admit *torsional* (twisting) resonances that can interleave with the higher *bending* resonances — a fact that becomes central to the v8.1 decision.

The data file is a `.npy` dictionary with 96,000 frequency points from 0 to 24 kHz at 0.25 Hz spacing. Only the band up to $\sim$ 5000 Hz is usable; above that the signal is noise. Six clear resonance peaks appear at approximately 45.8, 296.5, 830.5, 1633, 2708, and 4061 Hz.

2.2 Governing equation: the Euler–Bernoulli beam

For a beam with *constant* stiffness, harmonic transverse vibration $w(x,t) = W(x)\sin(\omega t)$ reduces the partial differential equation to the spatial ordinary differential equation

$$EI\,W''''(x) - \rho A\,\omega^2\,W(x) = 0. \tag{1}$$

Here $W(x)$ is the (complex) mode shape at frequency ω, primes denote $\partial/\partial x$, and EI is the bending stiffness.

2.3 Variable stiffness: the product rule

If E varies along the beam, $E = E(x)$, the bending moment is $M = E(x)I\,W''$, and differentiating it twice (product rule, applied twice) gives the *variable-coefficient* form actually enforced in the PINN:

$$\boxed{E(x)\,W'''' + 2\,E'(x)\,W''' + E''(x)\,W'' - \frac{\rho A}{I}\,\omega^2\,W = 0} \tag{2}$$

The two extra terms ($2E'W'''$ and $E''W''$) vanish only when E is constant. Capturing them is precisely what lets the method recover a *spatial profile* $E(x)$ rather than a single number.

2.4 Complex modulus and hysteretic damping

Damping is introduced through a *complex* modulus,

$$E^*(x) = E(x)\,(1 + j\eta), \tag{3}$$

where η is the hysteretic loss factor (a single scalar, assumed uniform in space). The imaginary part is what makes resonance peaks finite in height rather than infinite; physically it represents energy dissipation per cycle. With a complex modulus, equation (2) produces two residuals (a real part and an imaginary part), both of which must be driven to zero.

Why the damping factor is kept scalar, not a function

A single-point FRF does not carry enough information to identify a spatially varying damping field without severe non-uniqueness, and for a homogeneous metal strip physical damping is roughly uniform anyway. Making η a function would create identifiability problems — many different $\eta(x)$ would explain the same data equally well.

2.5 Boundary conditions

Clamped end ($x = 0$), with spring compliance. A perfect clamp would force $W(0) = 0$ and $W'(0) = 0$. Real fixtures are not perfectly rigid, so two springs model the compliance: a translational spring k_x and a rotational spring k_φ. The residuals are

$$E(0)I\,W''(0) + k_\varphi\,W'(0) = 0 \qquad \text{(rotational balance)} \tag{4}$$

$$E(0)I\,W'''(0) + k_x\,W(0) = 0 \qquad \text{(translational balance)} \tag{5}$$

As $k_x, k_\varphi \to \infty$ these recover the ideal rigid clamp.

Free end ($x = L$), with tip mass and unknown force.

$$E(L)I\,W''(L) = 0 \qquad \text{(zero bending moment)} \tag{6}$$

$$E(L)I\,W'''(L) + m_{\text{tip}}\,\omega^2\,W(L) + F = 0 \qquad \text{(tip inertia + applied force)} \tag{7}$$

The tip force amplitude F is unknown because, although the excitation velocity is known, the actual force transmitted to the tip is not.

2.6 From displacement to the measured velocity

The network predicts displacement W; the measurement is velocity. For harmonic motion $V = j\omega W$, so

$$V_{\text{real}} = -\omega\,W_{\text{imag}}, \qquad V_{\text{imag}} = +\omega\,W_{\text{real}}. \tag{8}$$

This conversion is applied before comparing to data.

2.7 The five unknowns and why a PINN

Unknown	Type	Meaning
$E(x)$	function	Young's modulus along the beam
η	scalar	hysteretic damping loss factor
k_x	scalar	translational clamp spring
k_φ	scalar	rotational clamp spring
F	scalar	tip force amplitude

Classical curve-fitting can identify the four scalars under the assumption that E is constant. It cannot identify a *function* $E(x)$. The PINN can, because the network represents $E(x)$ as a continuous field and the variable-coefficient PDE (2) ties that field to the data. This is the sole justification for the PINN approach over the classical method a colleague runs in parallel.

3 The Stable PINN Core (Unchanged Across Versions)

The following machinery is common to v6–v8.1. Understanding it once makes the version diffs easy to read.

3.1 Two networks

WNet maps $(x, \omega) \mapsto (W_{\text{real}}, W_{\text{imag}})$: the complex displacement field at any position and frequency. It is a multilayer perceptron (MLP), 4 hidden layers of 128 units, tanh activations.

ENet maps $x \mapsto E(x)$: the stiffness field. 3 hidden layers of 64 units, tanh, with a sigmoid output bounded to $[50, 400]$ GPa and its final layer initialised so $E(x)$ starts near $E_0 = 200$ GPa (steel).

Why tanh and not ReLU

The PDE requires up to the *fourth* spatial derivative of W. ReLU has a second derivative that is zero almost everywhere, which would make the physics loss identically zero and uninformative. tanh is smooth to all orders, so the high-order autograd derivatives are well defined.

3.2 Scalar parameters and reparameterisation

η, k_x, k_φ, F are trainable scalars. They must stay positive, so each is stored as an unconstrained raw value passed through a softplus, value = softplus(raw) $\times$ scale, with per-parameter scale factors $(1, 10^3, 10^1, 1)$ so the raw values live in a comparable numeric range. $E(x)$ is bounded differently — by a sigmoid inside ENet — because it must stay within a physical interval rather than merely be positive.

3.3 Collocation points vs. training points

Two distinct point sets:

- **Collocation points** — a 50×50 uniform grid in (x, ω) (2500 points). The PDE residual (2) is enforced here. These points fill the entire 2-D domain; they do not require any measurement.
- **Training (data) points** — ~ 377 points on the 1-D slice $x = L$ at the measured frequencies. The data loss compares predicted vs. measured velocity here.

Boundary residuals are evaluated at $x = 0$ and $x = L$ across the collocation frequencies.

3.4 Autograd derivatives and float64

Derivatives are taken by automatic differentiation with `create_graph=True`, chained to reach fourth order: $W_x \to W_{xx} \to W_{xxx} \to W_{xxxx}$.

Why float64 is mandatory

Each `grad` call accumulates rounding error. Chaining four of them in float32 lets that error grow until the physics residual is dominated by noise. float64 keeps the fourth-order derivative meaningful. The cost is real: see the runtime note in Section 7.

3.5 The two normalisations (these are different things)

Input normalisation (since v3, unchanged) maps network *inputs* into a well-conditioned range: $x \in [0, L] \mapsto [-1, 1]$ and $\omega \in [\omega_{\min}, \omega_{\max}] \mapsto [-1, 1]$. This keeps tanh activations out of saturation.

Loss normalisation (rebuilt in v8) divides each loss *output* by a physical scale so the three components are comparable in magnitude. The two operations are independent and both happen every step. Confusing them is a common source of error, so they are kept conceptually separate throughout.

3.6 Two-phase training

Phase 1 — Adam, 20,000 epochs, learning rate 10^{-3}, with a cosine-annealing-warm-restarts schedule (restarts at increasing intervals). Adam explores broadly.

Phase 2 — L-BFGS, a quasi-Newton method that uses an approximate inverse Hessian to take well-informed steps near a solution. It refines what Adam found.

L-BFGS discipline

L-BFGS only *polishes*. If Adam left the network in a bad place, L-BFGS entrenches it. The rule: do not start L-BFGS until the Adam data loss is comfortably below 1.0 (see Section 5.4 for what 1.0 means).

3.7 Checkpointing and the delete-before-restart rule

Training state (weights, scalar parameters, optimiser state, scheduler state, and the full per-epoch `history`) is saved to Google Drive periodically so a Colab disconnect can be resumed. Crucially, a checkpoint stores *training progress*, not the program: on a fresh runtime all Python objects must be rebuilt by running the setup cells before the loop can inject the saved state into them.

Delete checkpoints when the loss landscape changes

Whenever the loss function, the input features, or the frequency range change, the old optimiser state and weights are invalid in the new landscape. The checkpoints must be deleted before the first run of the modified code, or the resume logic will silently load incompatible state and corrupt training.

4 Version History: the Change Log

4.1 v6 — the preprocessing baseline (working)

v6 is the version at which *data preprocessing* finally converged. Earlier versions had repeatedly destroyed the signal:

- v3 smoothed magnitude and phase separately; phase wrapping destroyed the imaginary component.
- v4 smoothed real and imaginary parts but with a window that flattened the peaks, and uniform subsampling missed them entirely.
- v5 used aggressive peak-aware sampling with light smoothing; peak detection went wild and produced 7383 points.

v6 fixed this with the realisation that smoothing and training must be *decoupled*:

- Hard cutoff at 5000 Hz (above is noise).

- **No smoothing on training data** — the peaks are too narrow to survive any meaningful smoothing.
- Peak *detection* runs on a heavily smoothed $|V|$ in dB (Savitzky–Golay window 101), used only to *locate* peaks and estimate their widths, never for training values.
- Peak-aware sampling: $\sim$ 40 points clustered around each resonance, $\sim$ 8 around each anti-resonance, $\sim$ 100 sparse background points, all snapped to raw frequencies. Result: $\sim$ 377 training points sitting on the raw peak tops.

The v6 principle

Detect on a smoothed signal; train on the raw signal. Uniform subsampling at FRF scale flattens or misses sub-Hz peaks; dense sampling on raw peak tops preserves them. v6's preprocessing is treated as fixed from here on.

4.2 v7 — first identification attempt (failed)

v7 added three things on top of v6: empirical loss normalisation, a data-only warmup, and a log-magnitude data loss. It failed completely.

Observed failure. Normalised loss values looked tiny ($\sim 10^{-9}$) but were misleading. $E(x)$ pegged near the upper bound (400 GPa). The scalars never moved from their initial values. The predicted FRF was a smooth curve with small bumps where training points happened to cluster — no resonance structure at all.

Two diagnosed root causes.

1. *The empirical normalisation was meaningless.* It divided each loss by the loss of the *randomly initialised* network. Reaching 10^{-9} of that reference merely meant the network had shrunk its random-large output to physical magnitude — not that it had fit any peak. The reference encoded the initialisation, not the physics.
2. *Spectral bias produced a trivial solution.* A tanh MLP prefers smooth, low-frequency functions. The resonance peaks are sub-Hz wide — very high-frequency content in ω. The network found a smooth average instead. Once the output is approximately smooth and small, $W \approx 0$ trivially satisfies the homogeneous PDE, so no gradient reaches the scalars or boundary conditions.

4.3 v8 — three targeted fixes

v8 replaced v7's mechanisms with three principled ones.

Fix 1 — physical loss normalisation. Replace the random-initialisation reference with fixed scales computed once from the data and known parameters (full formulas in Section 5.1). After this, `data_loss = 0.1` literally means "10% of the signal energy is mis-fit", and the same number is comparable across any state of any version.

Fix 2 — random Fourier features on ω. Inject a fixed bank of sinusoidal features of the normalised frequency so the network has a built-in oscillatory basis, directly countering spectral bias (Section 5.2). x is left unencoded because $W(x)$ is comparatively smooth.

Fix 3 — curriculum learning by frequency. Instead of a data-only warmup, all three losses are active from the start, but the *data* points are revealed low-frequency-first in four stages (Section 5.3).

What v8 revealed. Training did not collapse silently this time — the honest normalisation made the behaviour legible. Stage 1 (modes 1–2) fit well, reaching `data_loss` ≈ 0.14. Stage 2 (adding mode 3) degraded to ≈ 0.39 but still fit. Stage 3 (adding modes 4–5 at 1633 and 2708 Hz) pushed the data loss to ≈ 1.25 — *above* the zero-output baseline, meaning the network abandoned its good fit. This is the diagnostic the v8 design anticipated.

4.4 v8.1 — reduce scope to the trustworthy modes (current)

The change. v8.1 is a small, surgical edit to v8: *reduce the set of modes the PINN tries to approximate* to the three unambiguous bending modes (1, 2, 3, at 45.8, 296.5, 830.5 Hz), excluding everything from ~ 1000 Hz upward. Concretely, the frequency cutoff is lowered to ~ 1200 Hz — a value that lies in the gap between mode 3 (830.5 Hz) and mode 4 (1633 Hz), so it keeps mode 3's full band while excluding the suspect higher modes.

Why this is the right move.

- The Stage-3 collapse coincides exactly with the frequency band the supervisor had independently flagged as possibly *torsional* rather than bending. A bending PDE cannot represent a torsional resonance; forcing it to fit one creates a contradiction that drives the network to the trivial solution (Section 5.7).
- Mode 6 (4061 Hz) is dropped too: it sits near the noise floor (low SNR approaching the 5000 Hz cutoff) and its bending-vs-torsional status is not independently confirmed. Including a noisy, unconfirmed peak injects noise into every identified parameter.
- Three clean bending modes are sufficient to identify the scalars and a low-spatial-resolution $E(x)$. For a homogeneous metal strip, $E(x)$ is nearly constant anyway.

What is *not* changed in v8.1. The Fourier features (`N_FOURIER=64`, `FOURIER_SIGMA=10`) are held fixed — this isolates the frequency-range reduction as the single changed variable. The architecture, the physical normalisation, and the two-phase training all stay as in v8.

Implementation note. Because the pipeline derives $\omega_{\min}$, $\omega_{\max}$, the normalisation, the peak detection, and the collocation grid from a single `MAX_FREQ_HZ` constant, lowering that constant to ~ 1200 cascades correctly through the whole notebook. A pleasant side effect: with a $\sim 4\times$ smaller frequency window, the renormalisation makes each physical peak roughly $4\times$ wider in normalised coordinates, so the existing Fourier features have correspondingly more resolution on them (Section 5.2).

Before running v8.1

Lowering `MAX_FREQ_HZ` changes the normalisation, hence the meaning of every network input. Delete the Adam, L-BFGS, and "done" checkpoints first so the run starts cleanly from epoch 0.

4.5 Summary table

Ver.	Key change	Outcome
v6	Detection-only smoothing; raw values for training; peak-aware sampling	Preprocessing converged (∼377 clean points)
v7	Empirical loss normalisation + data-only warmup + log-magnitude loss	Failed: meaningless normalisation; spectral-bias collapse to trivial solution
v8	Physical normalisation; random Fourier features on ω; frequency curriculum	Low modes fit; collapse when suspected-torsional modes entered at Stage 3
v8.1	Reduce modes to 1–3 (cutoff ∼1200 Hz); Fourier held fixed	Train only on trustworthy bending modes; higher modes become validation

5 Concepts Explained in Depth

5.1 Loss normalisation: empirical (wrong) vs. physical (right)

A PINN loss sums three terms whose raw magnitudes differ by enormous factors. The PDE residual involves $E \sim 10^{11}$ Pa and is naturally astronomically large; the data loss is tiny by comparison. Summed raw, the optimiser sees only the largest term. Normalisation rescales each term so the *weights* you assign actually reflect priorities.

v7's empirical normalisation (the mistake). It captured each term's value from one forward pass of the *randomly initialised* network and divided by that. The problem: a random network's output magnitude has nothing to do with the physics. Driving the normalised data loss to 10^{-9} only meant the network had shrunk a random-large output down to a physical scale. It told you nothing about peak fitting, and it made the loss numbers actively misleading.

v8's physical normalisation (the fix). Three fixed scales, computed once from data and known parameters — never from the network:

$$V_{\text{meas}}^2 = \overline{V_r^2 + V_i^2} \qquad \text{(mean measured energy)} \tag{9}$$

$$\omega_0 = \sqrt{\frac{E_0\, I}{\rho A L^4}} \qquad \text{(reference modal frequency)} \tag{10}$$

$$W_{\text{scale}} = \frac{\max |V|}{\omega_{\max}} \qquad \text{(reference displacement amplitude)} \tag{11}$$

$$\text{pde\_scale} = \frac{\rho A\, \omega_0^2\, W_{\text{scale}}}{I} \tag{12}$$

$$\text{bc\_force\_scale} = m_{\text{tip}}\, \omega_{\max}^2\, W_{\text{scale}} \tag{13}$$

Each residual is divided by its scale before squaring; moment-type boundary residuals are addi-

tionally divided by L (a moment is a force times a length). The data loss becomes

$$\text{data\_loss} = \frac{\overline{(V_r^{\text{pred}} - V_r)^2 + (V_i^{\text{pred}} - V_i)^2}}{V_{\text{meas}}^2}. \tag{14}$$

The payoff: an interpretable number

With physical normalisation, `data_loss` is the fraction of signal energy mis-fit. 1.0 is the value you get from predicting nothing; 0.1 is a 10% mis-fit; 0 is perfect. The same number means the same thing in v8, v8.1, and any future version — it is a property of the predictor, not of the initialisation.

5.2 Spectral bias and random Fourier features

Spectral bias. Standard MLPs learn low-frequency components of a target function first and resist high-frequency ones. Intuitively, smooth functions are "easy" for the network and sharp features are "hard". The resonance peaks here are extremely sharp — with a loss factor $\eta = 0.01$, a peak's quality factor is $Q \approx 1/\eta = 100$, so its full width at half maximum is FWHM $\approx f/Q$. For the 45.8 Hz mode that is about 0.46 Hz: a feature roughly 10^{-4} of the frequency window wide. An unaided MLP simply will not synthesise something that sharp, which is exactly the v7 failure.

The fix (Tancik et al., 2020). Map the (normalised) frequency through a bank of random sinusoids before the MLP sees it. Draw a fixed random matrix $B \in \mathbb{R}^{N\times 1}$ with entries $B_i \sim \mathcal{N}(0, \sigma^2)$, and form

$$\gamma(u) = \big[\sin(2\pi B u),\ \cos(2\pi B u)\big], \tag{15}$$

where $u = \text{norm\_omega}(\omega)$. This gives the network a ready-made oscillatory basis, so sharp features in ω become *easy* to express. The WNet input is then $[\text{norm\_x}(x),\ \gamma(u)]$, of dimension $1 + 2N$. With $N = 64$, that is 129 inputs.

FOURIER-SIGMA is a scale, not a count: a clarified misconception

A natural but wrong reading is that `FOURIER_SIGMA` is "the number of terms each frequency is broken into". It is not. The *count* is `N_FOURIER` (=64), which sets how many random frequencies are drawn (giving $2 \times 64 = 128$ features). `FOURIER_SIGMA` (=10) is the standard deviation of the distribution those frequencies are drawn from — it controls how *high*/fast the features oscillate, i.e. the sharpness the network can represent. The analogy: it is the bandwidth of a Gaussian (RBF) kernel. Small σ = a smooth, slowly-wiggling basis (broad features only); large σ = a sharp, fast-wiggling basis (narrow peaks, at the cost of also being able to fit noise). N is how many basis functions; σ is how spiky each one may be.

Why only ω, not x. The displacement field $W(x)$ along the beam is smooth for low modes, so no encoding is needed there; the velocity $V(\omega)$ has the sharp peaks. (Caveat revisited in Section 5.7: high bending modes have spatially oscillatory shapes, so for those, x *may* also need encoding.)

Interaction with the v8.1 range cut. Narrowing the window from 5000 Hz to $\sim$ 1200 Hz shrinks it about 4×. A fixed physical peak then occupies about 4× more of the normalised $[-1, 1]$ axis, so the same $\sigma = 10$ features resolve it about 4× better. This is why σ is held fixed in v8.1 — the range cut already buys resolution, and changing two things at once would muddy the diagnosis. If the modes 1–3 fit still plateaus loose, σ is the next single knob to raise — in a separate run.

5.3 Curriculum learning by frequency

Rather than confront the network with all six peaks at once, the data points are revealed in stages, low frequencies first, while the PDE and boundary losses stay full-range throughout. The v8 schedule was:

Stage	Epochs	Data kept	Modes
1	0–3000	$f \leq 500$ Hz	1–2
2	3000–8000	$f \leq 1500$ Hz	1–3
3	8000–15000	$f \leq 3000$ Hz	1–5
4	15000–20000	full	all 6

This complements the Fourier features: the network gets the easiest targets first and builds the harder structure on top of a good initialisation. Only the data loss is masked — the collocation grid (physics) does not depend on training points and stays full-range.

The curriculum doubles as a diagnostic

The design explicitly anticipated that if early stages fit cleanly but a later stage ruined the fit, the modes added at that stage are suspect. That is exactly what happened at Stage 3, and it is the evidence behind v8.1.

Under v8.1, with `MAX_FREQ_HZ`≈ 1200, the Stage 2/3 thresholds of 1500/3000 Hz become no-ops, so the curriculum naturally collapses to a clean two-phase schedule: modes 1–2 first, then all three. It may be simplified accordingly.

5.4 The trivial solution and the data-loss baseline of 1.0

The PDE (2) is homogeneous (right-hand side zero). Therefore $W \equiv 0$ satisfies it *exactly*, and it satisfies the homogeneous parts of the boundary conditions too. The only things that break the $W = 0$ symmetry are the data loss and the lone $+F$ term in the tip boundary condition.

This produces a characteristic failure signature when the network gives up:

- **data ≈ 1 (or slightly above).** With physical normalisation, predicting $W \approx 0$ gives data_loss ≈ 1 by construction. Values just above 1 occur when the masked region carries more energy than the global mean used in V^2_{meas}.
- **PDE and BC $\approx 10^{-5}$.** Tiny physics loss here is *bad* news — it means $W \approx 0$ is satisfying the equations for free, not that the physics has been learned.
- **η, k_x, k_φ frozen.** They enter the loss only through products with W and its derivatives; with $W \approx 0$ their gradient vanishes.
- **F drifting toward 0.** In the tip BC, F appears as a bare additive term; with $W \approx 0$ the condition just wants $F = 0$. A downward F drift is therefore a direct collapse indicator.

5.5 Diagnostic: collapse-from-start vs. fit-then-broke

When a run ends with data near 1, there are two very different explanations, distinguished by reading the per-epoch `history` stored in the checkpoint:

- **Collapse-from-start** — data never dropped below ~ 1 in any stage. The network output was ≈ 0 from the beginning. This points at *representation*: the basis cannot build a peak (a σ problem).
- **Fit-then-broke** — data dropped well below 1 in an early stage and a later stage destroyed it. This points at a *physics/model mismatch*, not representation.

The actual checkpoint history was unambiguous: `data_loss` reached 0.1417 at epoch 2999 (end of Stage 1) and degraded to 0.3884 in Stage 2 before collapsing in Stage 3. This is fit-then-broke.

A diagnosis that was revised by evidence

An earlier hypothesis blamed `FOURIER_SIGMA` being too small (a collapse-from-start explanation). The checkpoint history overturned it: the low modes *did* fit, so representation was not the blocker. The break coincided with the suspected-torsional band. Running the diagnostic instead of assuming is what corrected the course.

5.6 Bending vs. torsional modes and the frequency-ratio test

Bending and torsional cantilever resonances follow completely different frequency spacings, which lets you classify peaks from frequencies alone — independent of the PINN, and cross-checkable by classical curve-fitting.

Bending (clamped–free) frequencies scale with $(\beta_n L)^2$, where the first six eigenvalues are

$$\beta_n L = 1.875,\ 4.694,\ 7.855,\ 10.996,\ 14.137,\ 17.279,\ \ldots$$

giving the ratios in the table. **Torsion** follows odd integers $1 : 3 : 5 : 7 \ldots$

	1	2	3	4	5	6
Observed f (Hz)	45.8	296.5	830.5	1633	2708	4061
Observed ratio f/f_1	1.00	6.47	18.13	35.66	59.13	88.67
Bending ratio	1.00	6.27	17.55	34.38	56.84	84.92
Torsional ratio (odd)	1	3	5	7	9	11

The observed ratios track the *bending* series closely — all about 3–5% high, in a smooth monotonic trend consistent with base compliance and tip mass perturbing an ideal cantilever. On spacing alone, all six look like bending. This is precisely why the bending-vs-torsional question is genuinely ambiguous and could not be settled by assumption.

5.7 Two competing hypotheses for the Stage-3 break

Two explanations fit the evidence, with *different* fixes:

Hypothesis A — the higher modes are torsional. A bending PDE has no solution with a peak at a torsional resonance. The data loss demands a peak at 1633 Hz; the full-range bending physics forbids one. The contradiction's least-bad compromise is the trivial solution — the observed collapse. Fix: exclude those modes from the data (v8.1).

Hypothesis B — they are high-order *bending* modes the network cannot represent spatially. Mode 5 has four interior nodes; its shape oscillates rapidly in x. But x is *not* Fourier-encoded ("$W(x)$ is smooth" holds only for low modes), and a 50-point collocation grid is thin for such shapes. This is spectral bias in the *spatial* dimension. Fix: more spatial capacity (Fourier on x, deeper net, denser collocation), keeping the modes.

The frequency ratios lean toward B (all six fit a bending series); the supervisor's independent flag leans toward A. The honest position is that this is not yet resolved.

The path forward does not require resolving A vs. B first

Whatever modes 4–6 are, modes 1–3 are unambiguous bending: clean ratios, below the flagged band, already fit cleanly in the PINN, good SNR. v8.1 identifies on those. The highest-information step to settle A vs. B is to ask what the supervisor's torsional flag was based on (an FE model? hammer-test mode shapes?); FE-predicted torsional frequencies would settle it immediately.

5.8 Validation by forward-solving the identified model

Once $E(x), \eta, k_x, k_\varphi, F$ are identified from modes 1–3, the excluded modes become a held-out test. The correct way to use them is *not* to feed the trained network frequencies outside its training range (that is unreliable extrapolation, especially with Fourier features). Instead, take the identified parameters, *forward-solve* the Euler–Bernoulli problem numerically across the full range, and see where it predicts peaks. If it predicts a bending peak near 1633/2708 Hz, those modes are bending (Hypothesis B), and they can be re-included later with more spatial capacity. If it does not, they are not bending (Hypothesis A) and stay excluded. This turns the ambiguity into a test.

5.9 Self-Adaptive PINNs (SA-PINNs) — planned for v9

McClenny & Braga-Neto's Self-Adaptive PINNs replace fixed loss weights with *trainable, pointwise* weights. Each point i carries a weight λ_i, and the objective becomes a min–max:

$$\min_{\theta} \max_{\lambda} \frac{1}{N} \sum_i m(\lambda_i)\, r_i^2, \tag{16}$$

where r_i is the residual at point i, θ are network parameters, and $m(\cdot)$ is a non-negative mask function. The network minimises; the weights *ascend*. Any point whose residual stays stubbornly high has its λ pushed up, increasing its pull until the network must attend to it. It is a self-balancing attention mechanism over the residual field, and it retires the hand-tuned $(w_{\text{pde}}, w_{\text{bc}}, w_{\text{data}})$ weights.

Why it fits this problem. Fixed weights let the optimiser lower the *average* data loss by fitting easy off-peak regions and smoothing the peaks away — a few high-residual points barely move the mean. Per-frequency adaptive weights on the *data* term would grow at exactly the under-fit peaks and force the network to sharpen them, plausibly tightening the modes 1–3 fit from ~ 0.14 toward

0.01–0.05. (Here the leverage is on the data term, not the classical PDE/BC weighting, because the PDE/BC residuals are already tiny — the under-fit lives in the data.)

Why SA-PINNs must wait for clean data

SA-PINNs amplify weight wherever the residual is stubborn — *including residuals the model cannot satisfy*. If a torsional peak were still in the data, the method would pour unbounded weight into that impossible residual and destabilise training. So SA-PINNs are only safe *after* the data is restricted to modes the bending model can explain. The v8.1 reduction is a prerequisite, not merely a tidy-up. SA-PINNs also have their own knobs (λ learning rate, the mask m) and the min–max can oscillate, so they should be the single change in v9 — never bundled with another change.

6 Development Q&A Log

These are the substantive questions raised during development and how they were resolved. They are recorded because the reasoning is as valuable as the code.

Q1. After a Colab disconnect, do I rerun all cells or just the training loop?

A checkpoint stores training *progress* (weights, optimiser/scheduler state, history), not the program. On a fresh runtime every Python object is gone, and the resume logic injects saved state into objects that must already exist. So all setup cells must run first (including re-uploading the data); the training cell then auto-resumes from the Drive checkpoint — it does *not* retrain from scratch. If the kernel is still alive, just rerun the loop. If the loss function, features, or frequency range changed, delete the checkpoints first.

Q2. How long does training take on a Colab T4?

Roughly 45 minutes to 2 hours for the full run, but the dominant factor is that the T4's float64 throughput is about 1/32 of its float32 rate — the T4 is close to a worst-case GPU for this float64-bound workload — and the fourth-order autograd chain is inherently sequential (giving only a $\sim$ 3–5$\times$ speedup over CPU). The reliable approach is to time the first 100–200 epochs and extrapolate. For float64 PINNs an A100 or V100 is dramatically faster; the L4, despite being newer, shares the T4's crippled float64 and does not help.

Q3. Should I pause the collapsed run?

Yes. A representational/physics collapse does not recover with more epochs — a mid-run warm restart had already thrown a full learning-rate kick at it and it returned to the same basin. Pausing costs nothing diagnostically, because the history plot reads from the checkpoint, not the live run.

Q4. Is FOURIER_SIGMA the number of terms each frequency is broken into?

No — see the boxed clarification in Section 5.2. `N_FOURIER` is the count of random frequencies; `FOURIER_SIGMA` is the standard deviation of the distribution they are drawn from, i.e. a bandwidth/sharpness control, not a count.

Q5. Why include mode 6? The data is noisy there, and it might be torsional.

Both points were conceded. Mode 6 sits near the 5000 Hz noise floor (marginal SNR), and "it is above the flagged band, so it should be bending" was a weak argument — the flag is a hypothesis, not a measurement. Mode 6 is therefore dropped from the primary identification, and its status is settled empirically by forward-solve validation (Section 5.8), not assumed.

Q6. Are we just reducing the modes the PINN approximates? (v8.1)

Yes. v8.1 is exactly that: lower the cutoff to ~ 1200 Hz so the PINN fits only the three unambiguous bending modes, hold the Fourier features fixed, and use the excluded modes for validation. SA-PINNs remain a separate, later step (v9).

7 Practical Runbook

To implement v8.1.

1. Delete `CKPT_ADAM`, `CKPT_LBFGS`, `CKPT_DONE` from Drive.
2. Lower `MAX_FREQ_HZ` to $\sim$ 1200; let it cascade through preprocessing, normalisation, and the collocation grid.
3. Keep `N_FOURIER=64` and `FOURIER_SIGMA=10` unchanged.
4. Optionally simplify the curriculum to two effective stages (modes 1–2, then all three) and shorten the epoch budget.
5. Train Adam first; only start L-BFGS once the Adam data loss is below 1 (ideally well below).

What good v8.1 results look like.

- `data_loss` for modes 1–3 settles low (target ≤ 0.1, ideally lower).
- η, k_x, k_φ, F move from their initial values and stabilise.
- $E(x)$ leaves the upper bound and settles near ~ 200 GPa.
- The predicted FRF shows peaks at 45.8, 296.5, 830.5 Hz with phase approximately tracking.

If the fit plateaus loose. Raise `FOURIER_SIGMA` as the single next change, in its own run. Do not also change the curriculum or weights at the same time.

On the horizon. Forward-solve validation against modes 4–6; ask the supervisor for the basis of the torsional flag; v9 SA-PINNs on the clean data; later extension to other beam materials (steel, PTGF) with material-specific warm-start checkpoints.

8 Symbol and Term Glossary

Symbol / term	Meaning
$W(x)$	complex transverse displacement mode shape
V	velocity FRF, $V = j\omega W$
$E(x)$	spatially varying Young's modulus (function-valued unknown)
$E^*(x)$	complex modulus $E(x)(1 + j\eta)$
η	hysteretic damping loss factor (scalar)
k_x, k_φ	translational / rotational clamp spring stiffnesses
F	tip force amplitude
I	second moment of area, $bh^3/12$ (weak axis)
A	cross-section area, bh
Q	quality factor, $\approx 1/\eta$; sets peak sharpness FWHM $\approx f/Q$
WNet	network mapping $(x, \omega) \mapsto (W_r, W_i)$
ENet	network mapping $x \mapsto E(x)$
collocation points	(x, ω) grid where the PDE residual is enforced
training points	measured frequencies where the data loss is enforced
input normalisation	maps network inputs to $[-1, 1]$
loss normalisation	divides each loss term by a physical scale
`N_FOURIER`	number of random Fourier frequencies (count)
`FOURIER_SIGMA`	std. dev. of those frequencies (sharpness/bandwidth, not a count)
spectral bias	MLPs' tendency to learn smooth/low-frequency functions first
trivial solution	$W \equiv 0$, which satisfies the homogeneous PDE/BCs for free
data baseline 1.0	normalised data loss obtained by predicting nothing
SA-PINN	Self-Adaptive PINN: trainable pointwise loss weights via min–max

End of reference document.